Inventora Sistem Enterprise Resource Planning dan Pengadaan

TUGAS KELOMPOK BASIS DATA

Perancangan dan Implementasi Basis Data
Firebase Cloud Firestore



Anggota Kelompok

1. **Ichya Ulumiddiin (103112400076)**
2. **M. Zaky Mubarok (103112400073)**
3. **Lutfi Shidqi Mardian (103112400077)**
4. **Daffa Tsaqifna Fauztsany (103112400032)**
5. **Rizky Iffat Venardi (2311102301)**

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

2026

Daftar isi

1. Pendahuluan....3

2. Arsitektur Firebase Inventora....4

3. Modul dan penyimpanan aktual....5

4. Struktur koleksi dan dokumen....6

5. Lima dokumen contoh....9

6. Operasi Firebase SDK dan alur PO....13

7. Keamanan dan integritas data....16

8. Hasil pengujian dan kesesuaian proyek....20

Lampiran A. Eksekusi dan sumber proyek....22

1. Pendahuluan

Inventora menggunakan Firebase Cloud Firestore sebagai basis data dokumen dan Firebase Authentication untuk autentikasi pengguna. Laporan ini menjelaskan implementasi yang terkonfirmasi pada kode proyek: koleksi purchaseOrders dengan detail barang, penerimaan, dan pembayaran yang tertanam dalam dokumen PO. Lima dokumen contoh disertakan bersama operasi Firebase SDK dan hasil pengujian pada emulator lokal.

1.1 Tujuan

Tujuan laporan adalah mendokumentasikan struktur data Inventora secara akurat, memperlihatkan cara membentuk koleksi melalui penyimpanan dokumen, serta menjelaskan pembacaan dan perubahan data sesuai alur pengadaan. Contoh dapat dipelajari tanpa mengubah basis data cloud proyek.

1.2 Ruang lingkup

Cakupan utama adalah pembuatan PO, persetujuan atau penolakan, penerimaan barang, pencocokan penerimaan, dan pencatatan pembayaran simulasi. Data contoh terdiri atas lima dokumen dalam satu koleksi aktif. Modul inventori, SDM, payroll, dan keuangan lain tetap dibahas sesuai lokasi penyimpanannya saat ini; modul tersebut tidak diklaim sudah mempunyai koleksi Firestore.

1.3 Istilah pada basis data dokumen

Istilah	Pemakaian dalam Inventora
Koleksi	Wadah dokumen, yaitu purchaseOrders.
Dokumen	Satu PO dengan identitas unik pada path purchaseOrders/{poId}.
Field	Atribut dokumen, misalnya vendor, total, dan status.
Map dan array	grn dan paymentInfo berupa map; items dan lineItems berupa array map.
Security Rules	Aturan akses berdasarkan autentikasi, peran, nominal, dan transisi status.

Firestore menggunakan dokumen dan koleksi, sehingga pembuatan struktur dan pengisian data pada laporan ini ditunjukkan melalui Firebase SDK, bukan sintaks relasional. Koleksi terbentuk ketika dokumen pertama berhasil disimpan; tidak diperlukan deklarasi tabel terlebih dahulu.

2. Arsitektur Firebase Inventora

2.1 Komponen utama

Komponen	Peran yang ditemukan pada proyek
Antarmuka React	Menyusun PO, menampilkan daftar, menerima perubahan melalui listener, dan mengirim tindakan pengguna.
Firebase Authentication	Login Google; identitas akun dan custom claim role menentukan hak akses server.
Cloud Firestore	Penyimpanan dokumen purchaseOrders dan transaksi perubahan status.
Firestore Security Rules	Memeriksa setiap operasi client; akses koleksi lain ditolak secara bawaan.
Penyimpanan browser	Data demo, cache PO, inventori, SDM, payroll, dan beberapa modul pendukung.

2.2 Alur data

Pada sesi non-demo, PurchaseUnitView membuat dokumen PO menggunakan runTransaction. ProcurementView membaca koleksi purchaseOrders melalui onSnapshot. Perubahan persetujuan, penerimaan, dan pembayaran ditulis melalui persistPoTransition. Helper tersebut membaca versi dokumen terkini, memeriksa peran dan transisi, kemudian mengirim patch dalam transaksi.

Pada mode demo, PO memakai data browser dan tidak ditulis ke Firestore. Cache browser tidak menjadi bukti bahwa suatu dokumen ada di cloud. Laporan dan contoh pengujian membedakan sesi demo, emulator lokal, dan implementasi cloud.

2.3 Bentuk identitas dan relasi

Document ID menjadi identitas utama PO. Field id disimpan pula di dalam dokumen dan harus sama dengan poId pada path ketika dokumen dibuat. Relasi penerimaan menggunakan grn.poId yang menunjuk kembali ke PO induknya. items, grn.items, dan paymentInfo disimpan bersama dokumen PO agar data transaksi dapat dibaca sebagai satu kesatuan.

3. Modul dan penyimpanan aktual

3.1 Peta penyimpanan berdasarkan kode

Data atau modul	Penyimpanan aktual
Purchase order non-demo	Koleksi Firestore purchaseOrders; detail item, GRN, dan pembayaran tertanam.
Akun dan peran server	Firebase Authentication serta custom claim role; bukan koleksi pengguna yang dibuat aplikasi.
Inventori	localStorage, key inventora_inventory_v1.
Pegawai dan payroll	localStorage, key inventora_hr_employees_v2 dan inventora_hr_payroll_history_v2.
Invoice keuangan	localStorage, key inventora_finance_invoices_v1.
Langganan dan billing	localStorage; antara lain inventora_subscription_v2 dan inventora_subscription_invoices_v1.
Anggaran departemen	Array DEFAULT_DEPARTMENT_BUDGETS pada kode komponen.
Notifikasi dan riwayat transaksi UI	Penyimpanan browser; pembayaran utama PO juga tercatat dalam map paymentInfo.

3.2 Status blueprint

firebase-blueprint.json mencantumkan purchaseOrders dan activities. Akan tetapi, pemanggilan SDK yang aktif dan Security Rules proyek hanya mengizinkan purchaseOrders. Tidak ditemukan penulisan koleksi activities pada implementasi yang diperiksa; aksesnya ditolak oleh aturan bawaan. Blueprint tersebut diperlakukan sebagai rancangan, bukan bukti koleksi aktif.

3.3 Kompleksitas yang sudah tersedia

Kompleksitas Inventora terletak pada model dokumen bertingkat, empat peran pengguna, pengendalian perubahan status, pencocokan penerimaan, transaksi saat menyimpan, serta pencatatan pembayaran simulasi. Jumlah koleksi tidak mewakili seluruh kompleksitas aplikasi. Penambahan koleksi untuk modul lokal memerlukan integrasi dan aturan akses tersendiri sebelum dinyatakan terimplementasi.

4. Struktur koleksi dan dokumen

4.1 Hierarki dokumen

```
purchaseOrders/{poId}
  id, vendor, desc, date, deliveryDate
  department, subtotal, discountAmount
  taxAmount, withholdingTaxAmount, total
  status, color, threeWayMatchStatus
  items[]
    id, sku, name, qty, unitPrice, subtotal
  lineItems[]
    id, sku, name, qty, unitPrice, totalPrice
  grn (opsional)
    id, grnNumber, poId, receivedDate, receivedBy
    suratJalanNumber, status, inspectorNotes
    items[]
      sku, name, orderedQty, receivedQty
      condition, notes
  paymentInfo (opsional)
    paidAt, method, txId, reference, amount
    accountOrCard, receiptNo, payerName
  lastMutationId (opsional, metadata sinkronisasi)
```

grn dan paymentInfo adalah map, bukan subkoleksi. items dan lineItems adalah array map. lastMutationId merupakan metadata operasional pada helper antrean persetujuan; field tersebut tidak terdapat pada interface PurchaseOrder utama. Field grn dan paymentInfo tidak disertakan ketika PO baru dibuat.

4.2 Field dokumen PO

Field	Tipe	Makna atau ketentuan
id	string	Nomor PO; sama dengan document ID saat create.
vendor	string	Nama pemasok sebagai nilai pada dokumen.
desc	string	Deskripsi atau ringkasan pengadaan.

Field	Tipe	Makna atau ketentuan
date	string	Tanggal tampilan yang dibuat aplikasi; bukan Timestamp Firestore.
deliveryDate	string	Opsional pada tipe; formulir mengisi tanggal YYYY-MM-DD.
department	string	Opsional; salah satu dari empat departemen pada tipe proyek.
subtotal	number	Jumlah nilai item; diisi formulir ketika membuat PO.
discountAmount	number	Nominal diskon.
taxAmount	number	Nominal pajak yang disimpan.
withholdingTaxAmount	number	Nominal potongan pajak.
total	number	subtotal - diskon + pajak - potongan.
status	string	Status alur pengadaan sesuai aturan transisi.
color	string	Warna badge antarmuka.
threeWayMatchStatus	string	MATCHED, DISCREPANCY, PENDING_RECEIPT, atau PENDING_INVOICE pada tipe.
items	array map	Detail item utama; pada tipe bersifat opsional.
lineItems	array map	Format kompatibilitas yang juga ditulis formulir; tipe proyek masih any[].

4.3 Detail item dan penerimaan

Setiap items[] memuat id, sku opsional, name, qty, unitPrice, dan subtotal. lineItems[] menyalin nilai yang sama tetapi menggunakan totalPrice sebagai nilai baris. Relasi ke katalog berupa SKU string; proyek belum menyimpan DocumentReference ke koleksi produk.

Bagian GRN	Isi yang dipakai proyek
Identitas	id, grnNumber, dan poId berupa string. grn.poId harus sesuai dengan path PO.
Penerimaan	receivedDate, receivedBy, dan suratJalanNumber berupa string. inspectorNotes opsional.
Status	MATCHED, DISCREPANCY, atau PENDING pada tipe GoodsReceiptNote.
grn.items[]	sku opsional, name, orderedQty, receivedQty, condition, dan notes opsional.
condition	EXCELLENT, GOOD, DAMAGED, atau SHORTAGE.

4.4 Informasi pembayaran

paymentInfo menyimpan paidAt, method, txId, reference, accountOrCard, receiptNo, dan payerName sebagai string serta amount sebagai number. paidAt berisi string waktu ISO pada contoh. amount harus sama dengan total PO ketika pembayaran dicatat. Pembayaran pada aplikasi ini merupakan simulasi, bukan konfirmasi transfer dari bank.

4.5 Konsekuensi model dokumen

Satu PO dapat memuat banyak item, satu map GRN terkini, dan satu map pembayaran. Model saat ini belum menyediakan koleksi riwayat penerimaan atau pembayaran berulang. Vendor dan departemen disimpan sebagai string, sehingga tidak ada foreign key atau JOIN yang ditegakkan Firestore. Konsistensi lintas field bergantung pada transaksi, validasi aplikasi, dan Security Rules.

5. Lima dokumen contoh

5.1 Ringkasan data akhir

Lima PO berikut bersifat fiktif dan mengikuti nama field aplikasi. Semua PO mula-mula dibuat berstatus Pending Approval. Status akhirnya dibentuk melalui tindakan yang diizinkan rules, sehingga contoh penerimaan dan pembayaran tidak melewati tahapan persetujuan.

Document ID	Status akhir	Total
PO-2026-IT-DEMO01	Pending Approval	Rp999.000.000
PO-2026-OPS-DEMO02	Disetujui	Rp199.800.000
PO-2026-RND-DEMO03	Barang Diterima	Rp199.800.000
PO-2026-GA-DEMO04	Selesai	Rp93.240.000
PO-2026-IT-DEMO05	Ditolak	Rp133.200.000

Total nilai kelima PO adalah Rp1.625.040.000. Angka tersebut mencakup satu PO yang ditolak dan bukan realisasi belanja. Hanya PO keempat yang mempunyai paymentInfo, dengan pencatatan pembayaran simulasi sebesar Rp93.240.000.

5.2 Pembentuk lima dokumen

Kode JavaScript berikut berasal dari contoh-data.mjs dan membentuk seluruh field dokumen, termasuk array detail, map GRN, dan map pembayaran. Variabel pendingDocuments berisi data yang dapat dibuat menurut rules. finalDocuments adalah hasil akhir yang diharapkan setelah alur dijalankan. Data akhir lengkap juga tersedia dalam contoh-dokumen.json.

```
// Lima PO fiktif dengan field yang dipakai Inventora.
// Tidak membuka koneksi dan tidak menulis ke Firebase.
export const specifications = [
  {
    id: 'PO-2026-IT-DEMO01',
    vendor: 'NVIDIA Corp Indonesia',
    department: 'IT Infrastructure',
    sku: 'NV-H100-TC', name: 'NVIDIA H100 Tensor Core GPU',
    qty: 2, unitPrice: 450000000,
  },
```

```
  {
    id: 'PO-2026-OPS-DEMO02',
    vendor: 'Cisco Systems Indonesia',
    department: 'Operations & Logistics',
    sku: 'SW-10GBE-L3', name: '10GbE Network L3 Managed Switch',
    qty: 4, unitPrice: 45000000,
  },
  {
    id: 'PO-2026-RND-DEMO03',
    vendor: 'Dell EMC Indonesia',
    department: 'Engineering & R&D',
    sku: 'STR-NVME-8TB', name: '8TB NVMe Enterprise Storage',
    qty: 10, unitPrice: 18000000,
  },
  {
    id: 'PO-2026-GA-DEMO04',
    vendor: 'APC by Schneider Electric',
    department: 'HR & GA',
    sku: 'UPS-3KVA-RT', name: '3kVA Rackmount UPS',
    qty: 3, unitPrice: 28000000,
  },
  {
    id: 'PO-2026-IT-DEMO05',
    vendor: 'Dell EMC Indonesia',
    department: 'IT Infrastructure',
    sku: 'RS-W1-PRO', name: 'Rack Server Web Infrastructure',
    qty: 1, unitPrice: 120000000,
  },
];

export const pendingDocuments = specifications.map((row, i) => {
  const subtotal = row.qty * row.unitPrice;
  const taxAmount = Math.round(subtotal * 0.11);
  const item = {
```

```
    id: `li-demo-${i + 1}`, sku: row.sku, name: row.name,
    qty: row.qty, unitPrice: row.unitPrice, subtotal,
  };
  const { subtotal: itemTotal, ...legacy } = item;
  return {
    id: row.id, vendor: row.vendor,
    desc: `Pengadaan contoh ${row.sku}`,
    date: '21 Okt 2026', deliveryDate: '2026-10-28',
    department: row.department,
    subtotal, discountAmount: 0, taxAmount,
    withholdingTaxAmount: 0,
    total: subtotal + taxAmount,
    status: 'Pending Approval', color: 'orange',
    threeWayMatchStatus: 'PENDING_RECEIPT',
    items: [item],
    lineItems: [{ ...legacy, totalPrice: itemTotal }],
  };
});

export function receiptPatch(po) {
  return {
    status: 'Barang Diterima', color: 'blue',
    threeWayMatchStatus: 'MATCHED',
    grn: {
      id: `GRN-${po.id}`, grnNumber: `GRN-${po.id}`,
      poId: po.id, receivedDate: '2026-10-28',
      receivedBy: 'Petugas Gudang Contoh',
      suratJalanNumber: `SJ-${po.id}`,
      items: po.items.map(item => ({
        sku: item.sku, name: item.name,
        orderedQty: item.qty, receivedQty: item.qty,
        condition: 'GOOD', notes: 'Sesuai pesanan',
      })),
      status: 'MATCHED', inspectorNotes: 'Data uji fiktif',
```

```
    },
  };
}

export function paymentPatch(po) {
  return {
    status: 'Selesai', color: 'green',
    paymentInfo: {
      paidAt: '2026-10-29T03:00:00.000Z',
      method: 'Transfer bank (simulasi)',
      txId: `TX-${po.id}`, reference: 'DEMO-TRANSFER-04',
      amount: po.total, accountOrCard: 'DEMO-0004',
      receiptNo: `REC-${po.id}`,
      payerName: 'Inventora (simulasi)',
    },
  };
}

export const finalDocuments = pendingDocuments.map((po, i) => {
  if (i === 0) return po;
  if (i === 4) return { ...po, status: 'Ditolak', color: 'red'
    };
  const approved = { ...po, status: 'Disetujui', color: 'green'
    };
  if (i === 1) return approved;
  const received = { ...approved, ...receiptPatch(po) };
  if (i === 2) return received;
  return { ...received, ...paymentPatch(po) };
});
```

5.3 Tanggal dan nominal contoh

Tanggal penerimaan dan pembayaran pada fixture adalah tanggal skenario, bukan waktu transaksi nyata. Nominal menggunakan bilangan bulat rupiah agar contoh tidak bergantung pada pembulatan pecahan. Tarif 11 persen hanya mengikuti pengaturan contoh pada kode formulir; laporan ini tidak menetapkannya sebagai tarif pajak yang berlaku untuk setiap transaksi.

6. Operasi Firebase SDK dan alur PO

Potongan berikut menjelaskan inti eksekusi dari uji-firestore.mjs. Skrip lengkap menyiapkan client emulator dengan peran yang berbeda, menulis data contoh, memeriksa hasil, dan menutup koneksi. Identitas mock hanya berlaku di emulator; aplikasi cloud memerlukan login Firebase Authentication serta custom claim yang disetel oleh administrator tepercaya.

6.1 Koneksi emulator untuk pengujian

```
import { initializeApp } from 'firebase/app';
import {
  getFirestore, connectFirestoreEmulator,
  collection, doc, getDocs, query, where,
  runTransaction, updateDoc,
} from 'firebase/firestore';
import {
  pendingDocuments, receiptPatch, paymentPatch,
} from './contoh-data.mjs';

function makeClient(role) {
  const app = initializeApp({
    projectId: 'demo-inventora-tubes',
    apiKey: 'demo-api-key',
  }, role);
  const db = getFirestore(app);
  connectFirestoreEmulator(db, '127.0.0.1', 8085, {
    mockUserToken: { sub: `demo-${role}`, role },
  });
  return db;
}
const warehouse = makeClient('Warehouse Manager');
const finance = makeClient('Finance Manager');
```

6.2 Membentuk koleksi dan mengisi lima dokumen

```
for (const po of pendingDocuments) {
  await runTransaction(warehouse, async tx => {
    const ref = doc(warehouse, 'purchaseOrders', po.id);
    const existing = await tx.get(ref);
    if (existing.exists()) {
      throw new Error('Document ID sudah digunakan.');
    }
    tx.set(ref, po);
  });
}
```

tx.set menulis satu dokumen PO. Pada dokumen pertama, koleksi purchaseOrders otomatis muncul. Pengecekan existing mencegah data contoh menimpa PO yang sudah ada. Kelima transaksi dilakukan terpisah; kegagalan pada dokumen berikutnya tidak membatalkan dokumen sebelumnya. Untuk mengulang fixture, gunakan instance emulator tugas yang baru.

6.3 Persetujuan dan penolakan

```
for (const i of [1, 2, 3]) {
  const ref = doc(finance, 'purchaseOrders',
    pendingDocuments[i].id);
  await updateDoc(ref, {
    status: 'Disetujui', color: 'green',
  });
}
await updateDoc(doc(finance, 'purchaseOrders',
  pendingDocuments[4].id), {
  status: 'Ditolak', color: 'red',
});
```

6.4 Penerimaan dan pembayaran simulasi

```
for (const i of [2, 3]) {
  const po = pendingDocuments[i];
  await updateDoc(doc(warehouse, 'purchaseOrders', po.id),
    receiptPatch(po));
}
const paidPo = pendingDocuments[3];
await updateDoc(doc(finance, 'purchaseOrders', paidPo.id),
  paymentPatch(paidPo));
```

PO pertama dibiarkan menunggu persetujuan. PO kedua berhenti pada Disetujui. PO ketiga berhenti pada Barang Diterima dan memiliki grn. PO keempat mencapai Selesai serta memiliki grn dan paymentInfo. PO kelima berstatus Ditolak. Pada implementasi antarmuka, perubahan tersebut memakai persistPoTransition dengan transaksi pembacaan dan pembaruan dokumen terkini.

6.5 Membaca dan menyaring dokumen

```
const snapshot = await getDocs(
  collection(finance, 'purchaseOrders'));
const orders = snapshot.docs.map(item => ({
  ...item.data(), id: item.id,
}));

const completed = query(
  collection(finance, 'purchaseOrders'),
  where('status', '==', 'Selesai'));
const completedSnapshot = await getDocs(completed);
console.log(orders.length, completedSnapshot.size);
```

Hasil fixture adalah lima PO dan satu PO berstatus Selesai. ProcurementView menggunakan onSnapshot untuk pembaruan daftar secara langsung dan menghentikan listener saat komponen dilepas. Penghapusan PO melalui SDK ditolak oleh rules saat ini.

7. Keamanan dan integritas data

7.1 Peran dan alur yang diizinkan

Tindakan	Peran	Prasyarat utama
Baca PO	Super Admin, Finance Manager, Warehouse Manager	Pengguna terautentikasi dan memiliki claim role yang sesuai.
Buat PO	Super Admin atau Warehouse Manager	Pending Approval; total cocok dengan komponen; tanpa grn dan paymentInfo.
Setujui atau tolak	Super Admin atau Finance Manager	Berawal dari Pending Approval.
Catat penerimaan	Super Admin atau Warehouse Manager	Berawal dari Disetujui atau Selisih Penerimaan; status GRN dan pencocokan selaras.
Catat pembayaran	Super Admin atau Finance Manager	Barang Diterima, MATCHED, belum dibayar, dan nominal sama dengan total PO.
Hapus PO	Tidak diizinkan	allow delete bernilai false.

HR Manager merupakan peran aplikasi untuk modul SDM, tetapi tidak diizinkan membaca PO oleh rules saat ini. Matriks izin yang disimpan di browser tidak menggantikan custom claim dan tidak mengubah Security Rules server.

7.2 Security Rules aktual

Berikut aturan dari firestore.rules proyek. Pembungkusan baris hanya mengubah whitespace agar kode muat pada halaman. Tidak ada deployment atau perubahan aturan produksi dalam penyusunan laporan.

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {
    function hasRole(roles) {
      return request.auth != null && request.auth.token.get(
        'role', '') in roles;
```

```
}
function finance() { return hasRole(['Super Admin',
  'Finance Manager']); }
function warehouse() { return hasRole(['Super Admin',
  'Warehouse Manager']); }

match /purchaseOrders/{poId} {
  allow read: if hasRole(['Super Admin', 'Finance Manager',
    'Warehouse Manager']);
  allow create: if warehouse()
    && request.resource.data.id == poId
    && request.resource.data.status == 'Pending Approval'
    && request.resource.data.total is number &&
      request.resource.data.total > 0
    && request.resource.data.subtotal is number &&
      request.resource.data.subtotal > 0
    && request.resource.data.discountAmount >= 0
    && request.resource.data.taxAmount >= 0 &&
      request.resource.data.withholdingTaxAmount >= 0
    && request.resource.data.total ==
      request.resource.data.subtotal -
      request.resource.data.discountAmount
      + request.resource.data.taxAmount -
        request.resource.data.withholdingTaxAmount
    && !('paymentInfo' in request.resource.data) && !('grn'
      in request.resource.data);

  allow update: if (
    finance() && resource.data.status == 'Pending Approval'
    && request.resource.data.status in ['Disetujui',
      'Ditolak']
    && request.resource.data.diff(resource.data)
      .affectedKeys().hasOnly(['status', 'color',
      'lastMutationId'])
```

```
    ) || (
      warehouse() && resource.data.status in ['Disetujui',
        'Selisih Penerimaan']
      && request.resource.data.status in ['Barang Diterima',
        'Selisih Penerimaan']
      && request.resource.data.grn.poId == poId
      && ((request.resource.data.status == 'Barang Diterima'
        && request.resource.data.threeWayMatchStatus ==
        'MATCHED' && request.resource.data.grn.status ==
        'MATCHED')
        || (request.resource.data.status ==
          'Selisih Penerimaan' &&
          request.resource.data.threeWayMatchStatus ==
          'DISCREPANCY' && request.resource.data.grn.status
          == 'DISCREPANCY'))
      && request.resource.data.diff(resource.data)
        .affectedKeys().hasOnly(['status', 'color', 'grn',
        'threeWayMatchStatus'])
    ) || (
      finance() && resource.data.status == 'Barang Diterima'
      && resource.data.threeWayMatchStatus == 'MATCHED'
      && !('paymentInfo' in resource.data)
      && request.resource.data.status == 'Selesai'
      && request.resource.data.paymentInfo.amount ==
        resource.data.total
      && request.resource.data.paymentInfo.txId is string
      && request.resource.data.paymentInfo.receiptNo is string
      && request.resource.data.diff(resource.data)
        .affectedKeys().hasOnly(['status', 'color',
        'paymentInfo'])
    );
    allow delete: if false;
  }
```

```
    match /{document=**} { allow read, write: if false; }
  }
}
```

7.3 Batas validasi yang perlu dipahami

Rules memeriksa rumus total ketika create, membatasi field yang boleh berubah, mengendalikan urutan status, serta menyamakan nominal pembayaran dengan total PO. Rules belum memvalidasi seluruh struktur items dan grn.items, jumlah subtotal terhadap semua detail, setiap format tanggal, maupun seluruh properti wajib pada tipe TypeScript. Pemeriksaan jumlah dan kondisi penerimaan juga dilakukan pada antarmuka; kesesuaian label MATCHED saja bukan pemeriksaan server yang lengkap atas seluruh item.

Firestore tidak mempunyai foreign key bawaan. grn.poId diperiksa terhadap path PO oleh rules penerimaan, sedangkan SKU dan nama pemasok masih berupa string. Transaksi mencegah keputusan berdasarkan versi dokumen yang usang, tetapi tidak menggantikan validasi skema dan aturan bisnis yang belum ditulis. Hasil uji berikut berlaku pada skenario yang disebutkan.

8. Hasil pengujian dan kesesuaian proyek

8.1 Lingkungan pengujian

Pengujian dilakukan pada 21 Oktober 2026 melalui emulator Firestore lokal, project ID demo-inventora-tubes, endpoint 127.0.0.1:8085. Emulator memuat firestore.rules proyek yang sama dengan laporan. Client SDK menggunakan identitas mock Warehouse Manager, Finance Manager, HR Manager, dan client tanpa autentikasi. Tidak ada data cloud produksi yang dibaca atau ditulis oleh skrip pengujian ini.

8.2 Hasil pemeriksaan

No	Pemeriksaan	Hasil
1	Lima ID unik dan rumus nominal sesuai	Lulus
2	Gudang membuat lima dokumen lewat transaksi	Lulus
3	Keuangan menyetujui PO 2 sampai 4	Lulus
4	Keuangan menolak PO 5	Lulus
5	Gudang mencatat penerimaan PO 3 dan 4	Lulus
6	Keuangan mencatat pembayaran simulasi PO 4	Lulus
7	Kelima dokumen sama dengan data akhir contoh	Lulus
8	Filter status mengembalikan satu PO selesai	Lulus
9	Baca tanpa autentikasi ditolak	Lulus
10	Baca PO oleh HR Manager ditolak	Lulus
11	Gudang tidak boleh menyetujui PO	Lulus
12	Total baru yang tidak cocok ditolak	Lulus
13	Pembayaran sebelum penerimaan ditolak	Lulus
14	Nominal pembayaran tidak cocok ditolak	Lulus
15	Pembayaran kedua pada PO selesai ditolak	Lulus
16	Penghapusan PO ditolak	Lulus
17	Koleksi activities belum diizinkan rules	Lulus

No	Pemeriksaan	Hasil
18	Uji negatif tidak mengubah lima dokumen	Lulus

Seluruh 18 pemeriksaan berhasil. Operasi yang sengaja tidak diizinkan ditolak dengan permission-denied. Pemeriksaan terakhir memastikan penolakan tersebut tidak mengubah isi lima dokumen. Bukti terperinci tersedia pada hasil-uji.json dan data akhir pada contoh-dokumen.json.

8.3 Kesesuaian dengan proyek Inventora

Aspek	Kesesuaian
Platform	Cloud Firestore dan Firebase Authentication, sesuai modul firebase proyek.
Struktur	Satu koleksi aktif purchaseOrders; map dan array sesuai tipe serta formulir.
Data contoh	Lima dokumen fiktif, dibuat melalui SDK dan alur status yang diizinkan.
Keamanan	Rules aktual, peran server, nominal pembayaran, dan pembatasan transisi diuji.
Modul lokal	Inventori, SDM, payroll, dan modul lain dijelaskan menurut penyimpanan aktualnya.

8.4 Kesimpulan teknis

Implementasi basis data Inventora yang terkonfirmasi menggunakan model dokumen Firestore dengan data pengadaan tertanam. Kompleksitas alur pengadaan, peran, dan transaksi tetap ada walaupun jumlah koleksi aktif hanya satu. Dokumen ini menyajikan implementasi tersebut beserta lima data contoh yang sudah diuji, tanpa menyatakan modul lokal sudah tersinkronisasi ke Firebase atau menyatakan pembayaran simulasi sebagai transaksi bank nyata.

Lampiran A. Eksekusi dan sumber proyek

A.1 Berkas paket Firebase

Berkas	Kegunaan
contoh-data.mjs	Pembentuk lima dokumen dan patch penerimaan/pembayaran.
contoh-dokumen.json	Kelima dokumen akhir lengkap, termasuk field bertingkat.
uji-firestore.mjs	Skrip emulator: membuat data, menjalankan alur, dan memeriksa 18 skenario.
hasil-uji.json	Bukti pengujian beserta hash sumber data dan rules.
README.md	Langkah menjalankan contoh secara lokal.

Seluruh berkas berada dalam folder database/firestore pada proyek Inventora. Skrip uji mengunci project ID demo dan alamat loopback port 8085. Jangan mengarahkan identitas mock emulator ke aplikasi cloud.

A.2 Menjalankan contoh lokal

Gunakan Node.js, dependensi proyek yang sudah terpasang, Java 21, serta JAR emulator Firestore yang tersedia pada folder .firebase-emulators. Mulai emulator tugas yang baru dengan firestore.rules proyek, project ID demo-inventora-tubes, dan port 8085. Petunjuk PowerShell lengkap tersedia di README paket.

```
node database/firestore/uji-firestore.mjs
```

Perintah tersebut menjalankan pengisian dan verifikasi, lalu menghasilkan ulang contoh-dokumen.json serta hasil-uji.json. Proses berhenti bila dokumen contoh sudah ada, agar tidak menimpa data. Setelah selesai, hentikan hanya instance emulator tugas.

A.3 Sumber implementasi yang diperiksa

Sumber proyek	Bukti yang digunakan
src/firebase.ts	Inisialisasi Cloud Firestore dan Firebase Authentication.
src/types.ts	PurchaseOrder, POLineItem, GoodsReceiptNote, dan paymentInfo.

Sumber proyek	Bukti yang digunakan
src/components/PurchaseUnitView.tsx	Pembuatan PO dan bentuk payload items serta lineItems.
src/components/ProcurementView.tsx	Listener purchaseOrders, cache, dan penyimpanan transisi.
src/lib/purchaseOrderStore.ts	Transaksi perubahan PO dan pemeriksaan custom claim.
src/lib/procurement.ts	Validasi transisi, nominal, dan normalisasi data pembayaran.
firestore.rules	Hak akses dan aturan perubahan dokumen.
firebase-blueprint.json	Rancangan purchaseOrders dan activities yang dibandingkan dengan implementasi.

Struktur dokumen ditetapkan berdasarkan kode yang dapat diperiksa. Isi Firestore cloud tidak diekspor; seluruh contoh pada laporan adalah data sintetis. Logo dan identitas kelompok mengikuti dokumen referensi yang diberikan.